```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Here I have imported all the necessary libraries required for this case study


#As we are working on Netflix data, I have downloaded and uploaded the netflix.csv
#To read this csv. I am using the below command
#!gdown 1hp16gnNzavLBTTGceuV3Xyvgr8_UUPoe
!pip install gdown==4.4.0
#!wget https://drive.google.com/file/d/1hp16gnNzavLBTTGceuV3Xyvgr8_UUPoe/view?usp=drive_link -O netflix.csv
!gdown 1hp16gnNzavLBTTGceuV3Xyvgr8_UUPoe
```

```
Requirement already satisfied: gdown==4.4.0 in /usr/local/lib/python3.11/dist-packages (4.4.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from gdown==4.4.0) (3.18.0)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.11/dist-packages (from gdown==4.4.0) (2.32.3)
Requirement already satisfied: six in /usr/local/lib/python3.11/dist-packages (from gdown==4.4.0) (1.17.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from gdown==4.4.0) (4.67.1)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.11/dist-packages (from gdown==4.4.0) (4.13.4)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/dist-packages (from beautifulsoup4->gdown==4.4.0) (2.7)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.11/dist-packages (from beautifulsoup4->gdown==4.4
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests[socks]->gdown==4.4
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests[socks]->gdown==4.4.0) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests[socks]->gdown==4.4.0) (2
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests[socks]->gdown==4.4.0) (2
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.11/dist-packages (from requests[socks]->gdown==4.4.6
Downloading...
From: https://drive.google.com/uc?id=1hp16gnNzavLBTTGceuV3Xyvgr8_UUPoe
To: /content/netflix.csv
100% 3.40M/3.40M [00:00<00:00, 29.3MB/s]
```

```
df = pd.read_csv('netflix.csv')
df
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a Cape Town t...
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Act...	To protect his family from a powerful drug lor...
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA	1 Season	Docuseries, Reality TV	Feuds, flirtations and toilet talk go down amo...
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, Romantic TV Shows, TV ...	In a city of coaching centers known to train I...
...	...	...	...	...	...	...	...	...	...	...	...	...
8802	s8803	Movie	Zodiac	David Fincher	Mark Ruffalo, Jake Gyllenhaal, Robert Downey J...	United States	November 20, 2019	2007	R	158 min	Cult Movies, Dramas, Thrillers	A political cartoonist, a crime reporter and a...
8803	s8804	TV Show	Zombie Dumb	NaN	NaN	NaN	July 1, 2019	2018	TV-Y7	2 Seasons	Kids' TV, Korean TV Shows, TV Comedies	While living alone in a spooky town, a young g...
8804	s8805	Movie	Zombieland	Ruben Fleischer	Jesse Eisenberg, Woody Harrelson,	United States	November 1, 2019	2009	R	88 min	Comedies, Horror Movies	Looking to survive in a world taken

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
#To get the shape of this data set.
#It tells the no of rows and columns
df.shape
```

```
(8807, 12)
```

From this above data, I can infer that,

1. There are 12 columns and 8807 rows in this data set.
2. There are 11 columns with object data type and 1 column with int data type.
3. Showid is acting as an index
4. This data is a combination of both Movies and TV shows
5. Columns like director, cast, country, listed_in, description have multiple values separated by commas

6. There are null values in columns like director, cast, country, rating, duration

7. Column duration has different values for movies and TV shows. For movies, it is in minutes and for TV shows, it is in seasons.

8. Column date_added has date and month values separated by commas.

9. Column release_year has only year values and not any dates

10. Column rating has values like TV-MA, TV-14, TV-PG, TV-Y, TV-Y7, TV-G, PG-13, PG, R, NR, G, TV-Y7-FV, NC-17, UR

11. Column type has values like Movie and TV Show.

12. Column listed_in has values like International Movies, Dramas, Comedies, Action & Adventure, Thrillers, Romantic Movies, Music & Musicals, Horror Movies, Sci-Fi & Fantasy, Children & Family Movies, Documentaries, Independent Movies, Classic Movies, Anime Features, Sports Movies, LGBTQ Movies,

13. Column country has values like United States, India, United Kingdom, Japan, South Korea, Canada, Spain, France, Egypt, China, Taiwan, Belgium, Brazil, Australia, Germany, Mexico, South Africa, Nigeria, Indonesia, Turkey, Sweden, Colombia, Czech Republic, Lebanon, Italy, Denmark, Singapore

```
df.describe()                    # Range and outliers
#This tells the range and outliers and count , mean, Standard deviation, min, max for the column release_year
```

	release_year
count	8807.000000
mean	2014.180198
std	8.819312
min	1925.000000
25%	2013.000000
50%	2017.000000
75%	2019.000000
max	2021.000000

```
#To get the info of this dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   show_id       8807 non-null   object
 1   type          8807 non-null   object
 2   title         8807 non-null   object
 3   director      6173 non-null   object
 4   cast          7982 non-null   object
 5   country       7976 non-null   object
 6   date_added    8797 non-null   object
 7   release_year  8807 non-null   int64 
 8   rating        8803 non-null   object
 9   duration      8804 non-null   object
 10  listed_in     8807 non-null   object
 11  description   8807 non-null   object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB
```

```
#From this we can infer that show_id, type, title, release_year, listen_in and description is not having any null values
#So we can see there are null values in other columns like director, cast etc..
# Also, We can find the number of null values for
df.isna().sum()
```

	0
show_id	0
type	0
title	0
director	2634
cast	825
country	831
date_added	10
release_year	0
rating	4
duration	3
listed_in	0
description	0

From the above df.isnull().sum(), we can infer that first we need to clean the data and fill the null values.

Firstly we have to replace the NaN values with categorical columns like Country as unknown country For columns like duration, NaN values should be filled with 0

```
#To modify the columns with Null values and the columns like director, for eg: if a movie is directed by 2 or more directors ,
#Here i have made it like for eg movie ABC is directed by directors Amar, Akbar and Anthony
#To clean this data i have made it like movie
# ABC - Amar, ABC- Akbar, ABC - Anthony
#Just to work on the clean data and to get an effective and  meaningful insights



# 1. Director Column
df['director'] = df['director'].fillna('Unknown Director')
df['director'] = df['director'].apply(lambda x: x.split(", ") if isinstance(x, str) else [])
df = df.explode('director').reset_index(drop=True)

# 2. Cast Column
df['cast'] = df['cast'].fillna('Unknown Cast')  # FIXED: inplace=True returns None
df['cast'] = df['cast'].apply(lambda x: x.split(", ") if isinstance(x, str) and x != 'Unknown Cast' else [x])
df = df.explode('cast').reset_index(drop=True)

# 3. Country Column
df['country'] = df['country'].fillna('Unknown Country')
df['country'] = df['country'].apply(
    lambda x: [i.strip() for i in x.split(',')] if isinstance(x, str) else ['Unknown Country'])
df = df.explode('country').reset_index(drop=True)
df['country'] = df['country'].replace('', 'Unknown Country')

# 4. Rating Column
df['rating'] = df['rating'].fillna('Unknown Rating')
df['rating'] = df['rating'].apply(lambda x: x.split(", ") if isinstance(x, str) else [x])
df = df.explode('rating').reset_index(drop=True)

# 5. Duration Column (you may want to standardize it later)
df['duration'] = df['duration'].fillna('0')

# 6. Listed_in (Genre)
df['listed_in'] = df['listed_in'].fillna('Unknown Genre')
df['listed_in'] = df['listed_in'].apply(lambda x: x.split(", ") if isinstance(x, str) else [x])
df = df.explode('listed_in').reset_index(drop=True)

# 7. Date Added
df['date_added'] = df['date_added'].fillna('Unknown Date')

# 8. Title
df['title'] = df['title'].fillna('Unknown Title')

df.shape
```

```
(202065, 12)
```

After cleaning all the columns we found that there are 20265 rows and 12 columns

```
#Just to verify, are we still finding any null values
df.isna().sum()
```

	0
show_id	0
type	0
title	0
director	0
cast	0
country	0
date_added	0
release_year	0
rating	0
duration	0
listed_in	0
description	0

From this, it is clear that our data is cleaned and we are ready to work on it.

```
#From the above cleaned data, we can see there is a combination of duration and seasons for TV shows and movies.
#So we need to cluster these data on duration column
df['Movie_Mins'] = df[df['type'] == 'Movie']['duration'].apply(lambda x: int(x.split(" ")[0])if isinstance(x, str) else 0)
df['Show_Seasons'] = df[df['type'] == 'TV Show']['duration'].apply(lambda x: int(x.split(" ")[0])if isinstance(x, str) else 0)
df
```

```
df
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	Unknown Cast	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm..
1	s2	TV Show	Blood & Water	Unknown Director	Ama Qamata	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows	After crossing paths at a party, a Cape Town t..
2	s2	TV Show	Blood & Water	Unknown Director	Ama Qamata	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	TV Dramas	After crossing paths at a party, a Cape Town t..
3	s2	TV Show	Blood & Water	Unknown Director	Ama Qamata	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	TV Mysteries	After crossing paths at a party, a Cape Town t..
4	s2	TV Show	Blood & Water	Unknown Director	Khosi Ngema	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows	After crossing paths at a party, a Cape Town t..
...	...	...	...	...	...	...	...	...	...	...	...	..
202060	s8807	Movie	Zubaan	Mozez Singh	Anita Shabdish	India	March 2, 2019	2015	TV-14	111 min	International Movies	A scrappy but poor boy worms his way into a ty..
202061	s8807	Movie	Zubaan	Mozez Singh	Anita Shabdish	India	March 2, 2019	2015	TV-14	111 min	Music & Musicals	A scrappy but poor boy worms his way into a ty..
202062	s8807	Movie	Zubaan	Mozez Singh	Chittaranjan Tripathy	India	March 2, 2019	2015	TV-14	111 min	Dramas	A scrappy but poor boy worms his way into a ty..
202063	s8807	Movie	Zubaan	Mozez Singh	Chittaranjan Tripathy	India	March 2, 2019	2015	TV-14	111 min	International Movies	A scrappy but poor boy worms his way into a ty..
202064	s8807	Movie	Zubaan	Mozez Singh	Chittaranjan Tripathy	India	March 2, 2019	2015	TV-14	111 min	Music & Musicals	A scrappy but poor boy worms his way into a ty..

202065 rows × 14 columns

```
#After altering the duration column, checking the shape
df.shape
```

```
(202065, 14)
```

Now 2 extra rows are added. One for Movie and other for TV shows

```
#Now the above data is cleaned
#Now we can infer the count of movies and tv shows
df['type'].value_counts()
```

type	count
Movie	145917
TV Show	56148

So from this we can get to know that, in the provided data. There are 145917 Movies and 56148 TV shows. But these are not unique value counts. To get the unique count.We need to use nunique()

```
#To get unique count of each column
df.nunique()
```

	0
show_id	8807
type	2
title	8807
director	4994
cast	36440
country	123
date_added	1768
release_year	74
rating	18
duration	221
listed_in	42
description	8775
Movie_Mins	206
Show_Seasons	15

```
#Now we have these columns in our data
df.columns
```

```
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
       'release_year', 'rating', 'duration', 'listed_in', 'description',
       'Movie_Mins', 'Show_Seasons'],
      dtype='object')
```

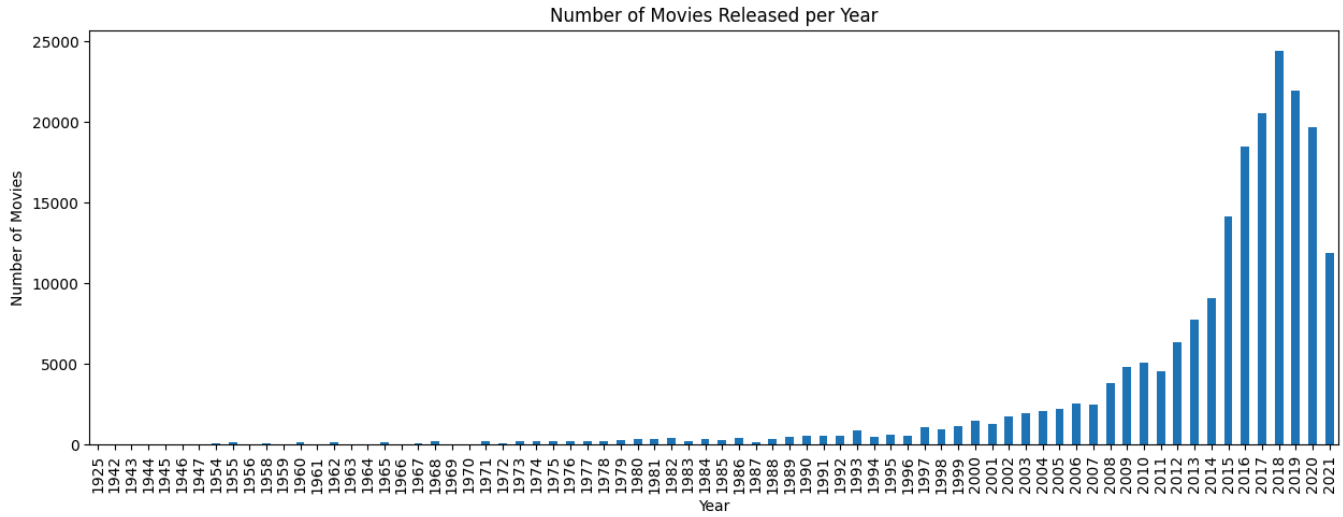
```
#Here is my first question
# 1.  How has the number of movies released per year changed over the last 20-30 years?
df['release_year'].value_counts()

# This problem is defined to know the present trend and comparing with the previous decades to analyse the releasing count.
```

release_year	count
2018	24441
2019	21931
2017	20516
2020	19697
2016	18465
...	...
1947	8
1942	6
1946	6
1943	5
1925	1

74 rows × 1 columns

From this we can infer that, It is highest in 2018 and slightly reducing in 2019 and 2020 and 2021

```
#The same question can be made graphical for more clear visualization
df['release_year'].value_counts().sort_index().plot(kind='bar', figsize=(15, 5))
plt.title('Number of Movies Released per Year')
plt.xlabel('Year')
plt.ylabel('Number of Movies')
plt.show()
```



From this we can infer that,

1. Count of movies has increased over the years.
2. Highest in the year 2018.
3. Found decreasing after 2018 till 2021.
4. We can also observe that, from the years till 2008 the movie released is less than 1000.
5. From the data, count for the recent year is almost closer to 5000 movies released in 2021.

```
# 2. Comparison of tv shows vs. movies.
#From this we will get to know the count of movies and tv shows
df['type'].value_counts()
```

type	count
Movie	145917
TV Show	56148

From This we can infer that,

1. Movie shows is highest compared to the TV shows

```
# To get the unique movies details
df[df['type'] == 'Movie'].nunique()
```

	0
show_id	6131
type	1
title	6131
director	4778
cast	25952
country	118
date_added	1533
release_year	73
rating	18
duration	206
listed_in	20
description	6105
Movie_Mins	206
Show_Seasons	0

```
#To get the unique count of TV shows details
df[df['type'] == 'TV Show'].nunique()
```

	0
show_id	2676
type	1
title	2676
director	300
cast	14864
country	66
date_added	1052
release_year	46
rating	10
duration	15
listed_in	22
description	2672
Movie_Mins	0
Show_Seasons	15

```
# To get the duration of movies count and Tv shows count
df.groupby('type')['duration'].value_counts()
```

type	duration	count
Movie	94 min	4343
	106 min	4040
	97 min	3624
	95 min	3560
	96 min	3511
...	...	...
TV Show	13 Seasons	132
	12 Seasons	111
	15 Seasons	96
	11 Seasons	30
	17 Seasons	30

221 rows × 1 columns

From the given data netflix, we can infer that,

1. Maximum duration of a movie is 94 mins
2. Count of movies whith the maximum duration is 1793
3. Maximum seasons of TV shows is 13 seasons
4. Count of Tv shows with 13 seasons is 65

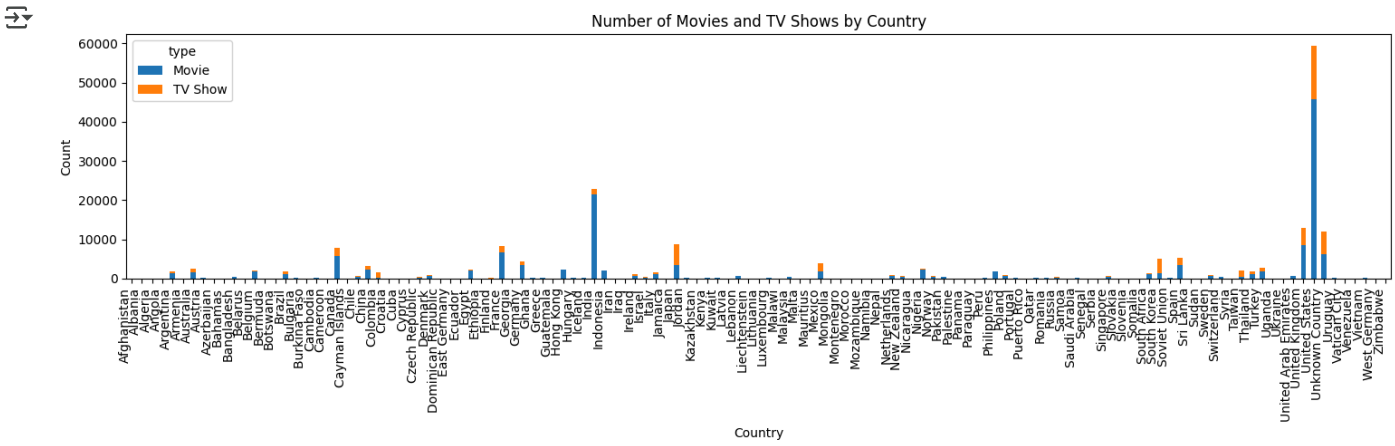
```
# We can also analyze the trend of movie and Tv show across the countries
top_countries = df['country'].value_counts().index
filtered_df = df[df['country'].isin(top_countries)]
filtered_df.groupby(['country', 'type']).size().unstack().fillna(0).astype(int)
```

country	type Movie	TV Show
Afghanistan	2	0
Albania	8	0
Algeria	77	0
Angola	32	0
Argentina	1349	455
...	...	...
Vatican City	3	0
Venezuela	28	0
Vietnam	134	0
West Germany	64	27
Zimbabwe	42	0

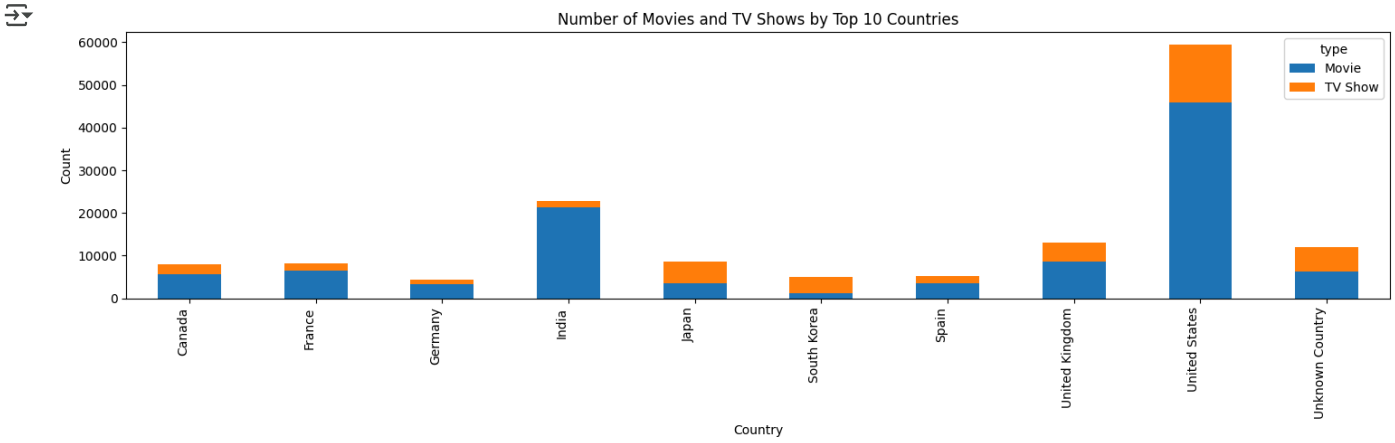
123 rows × 2 columns

From the above analysis of Movies and Tv shows we can infer the count of movies and tv shows accoding to the country. This can be best visualised using the graph

```
# To see the above trend of counts of movies and shows accross all the countries
countries = df['country'].value_counts().index
filtered_df = df[df['country'].isin(countries)]

filtered_df.groupby(['country', 'type']).size().unstack().fillna(0).astype(int).plot(kind='bar', stacked=True, figsize=(15, 5))
plt.title('Number of Movies and TV Shows by Country')
plt.xticks(rotation=90, ha='right')
plt.xlabel('Country')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```




```
#The above graph is too wide, so to make it easier and readable, I am choosing only top 10 countries
top10_countries = df['country'].value_counts().head(10).index
filtered_df = df[df['country'].isin(top10_countries)]

filtered_df.groupby(['country', 'type']).size().unstack().fillna(0).astype(int).plot(kind='bar', stacked=True, figsize=(15, 5))
plt.title('Number of Movies and TV Shows by Top 10 Countries')
plt.xticks(rotation=90, ha='right')
plt.xlabel('Country')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```



From the above pictorial representation, We can infer that

1. United States has topped in both Movies and TV shows
2. Then comes India considering only the Movies
3. Third place is for the Unknown Country for the Movies
4. TV shows are high in US > Unknown Country > UK > Japan and so on
5. Both Movies are TV shows are highly released in United states compared to any other countries.

```
# 3.What is the best time to launch a TV show?
# It will more helpful if the question is more precise. As it is asking for time, We cmay consider this as a monthwise, datewise, daywis
#As the requirement is not clear, I choose based on date and month and grouped accordingly
df.groupby('date_added')['type'].value_counts()
```

		count
date_added	type	
April 15, 2018	TV Show	2
April 16, 2019	TV Show	12
April 17, 2016	TV Show	18
April 20, 2017	TV Show	18
April 4, 2017	TV Show	33
...	...	...
September 9, 2020	Movie	66
	TV Show	4
September 9, 2021	Movie	40
	TV Show	7
Unknown Date	TV Show	158

2585 rows × 1 columns

From this we can Infer that, the movies or tv shows added based on the date. But to understand more about the particular date where more no of movies or shows are added. We need to do small changings to this

```
df.groupby('type')['date_added'].value_counts()
```

		count
type	date_added	
Movie	January 1, 2020	3470
	July 1, 2021	1913
	November 1, 2019	1899
	October 1, 2017	1730
	March 1, 2018	1696
...	...	...
TV Show	March 15, 2020	1
	May 11, 2021	1
	November 18, 2020	1
	September 22, 2020	1
	September 28, 2020	1

2585 rows × 1 columns

From this we get to know the highest no of movies added on January 1, 2020 and that is 3470. Best time is in the month of Jan. Both the ways are correct. It all depends on the question and the requirement. Based on that we can use according to our need. To find this in more detail, like on which day or date. We can also do this

```
df['date_added'] = pd.to_datetime(df['date_added'],  errors='coerce')
df['month_added'] = df['date_added'].dt.month
df['day_added'] = df['date_added'].dt.day


#Movie and TV shows added per month counts
df.groupby('month_added')['type'].value_counts()
```

month_added	type	count
1.0	Movie	13947
	TV Show	3941
2.0	Movie	9137
	TV Show	3786
3.0	Movie	11507
	TV Show	4201
4.0	Movie	12538
	TV Show	4460
5.0	Movie	9579
	TV Show	4111
6.0	Movie	11616
	TV Show	4959
7.0	Movie	15075
	TV Show	5129
8.0	Movie	11924
	TV Show	5029
9.0	Movie	13220
	TV Show	4818
10.0	Movie	13541
	TV Show	4199
11.0	Movie	11065
	TV Show	4428
12.0	Movie	12768
	TV Show	5341

From this we can get to know the count of movies and Tv shows as per the month over the years.

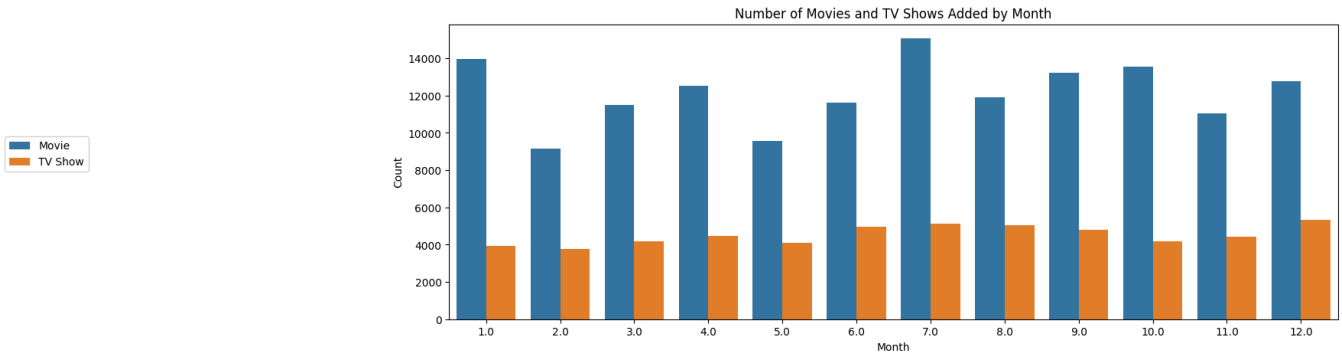
```
# if we want to read data based on both date and month
df.groupby(['month_added', 'day_added'])['type'].value_counts()
```

month_added	day_added	type	count
1.0	1.0	Movie	6726
		TV Show	1458
	2.0	Movie	126
		TV Show	50
	3.0	Movie	110
...	...	...	...
12.0	29.0	Movie	91
	30.0	TV Show	222
		Movie	113
	31.0	Movie	1929
		TV Show	401

724 rows × 1 columns

From this we get to know the the count of each date and that month. And we can note that highest no of movies are added on Jan 1st As it is a New Year. So we can analyse that best time to add movies or TV shows is on Holidays.

```
# This can be visualized with the help of graph
plt.figure(figsize=(15, 5))
sns.countplot(x='month_added', hue='type', data=df)
plt.title('Number of Movies and TV Shows Added by Month')
plt.xlabel('Month')
plt.ylabel('Count')
plt.legend(loc =(-0.5, 0.5))
plt.show()
```



From this we can infer that, highest movies are added in July month and then comes Jan. Whereas Tv shows are slightly higher in August than July. Lowest in the month of February

```
# 4. Analysis of actors/directors of different types of shows/movies.
#To get to know how many times each director has worked with particular cast
df.groupby('director')['cast'].value_counts()
```

		count
director	cast	
A. L. Vijay	G.V. Prakash Kumar	3
	Hema	3
	Joy Mathew	3
	Munishkanth	3
	Murli Sharma	3
...	...	...
Şenol Sönmez	Seda Güven	3
	Somer Karvan	3
	Yosi Mizrahi	3
	Zerrin Sümer	3
	Özgür Emre Yıldırım	3

62741 rows × 1 columns

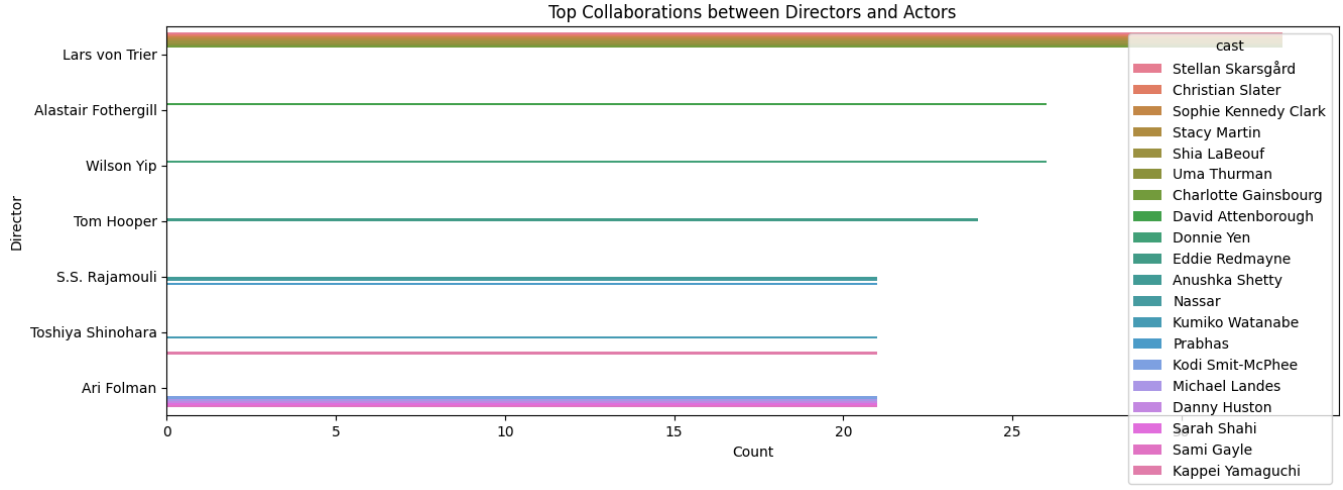
```
#To get to know the director and the cast and what type of genre they worked in
df.groupby(['director', 'cast'])['listed_in'].value_counts()
```

director	cast	listed_in	count
A. L. Vijay	G.V. Prakash Kumar	Comedies	1
		Dramas	1
		International Movies	1
	Hema	Comedies	1
		International Movies	1
...	...	...	...
Şenol Sönmez	Zerrin Sümer	Dramas	1
		International Movies	1
	Özgür Emre Yıldırım	Comedies	1
		International Movies	1
		Romantic Movies	1

148260 rows × 1 columns

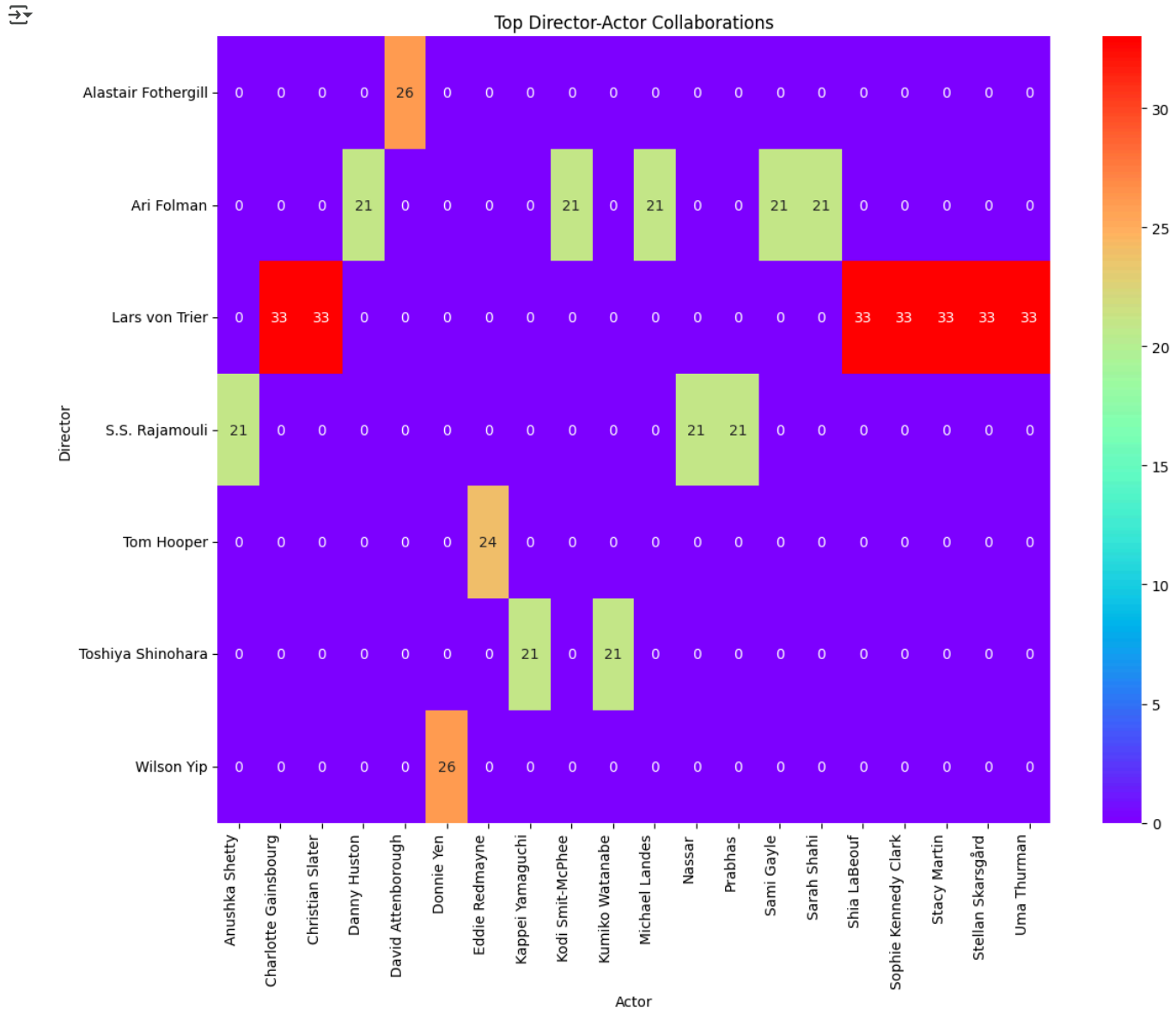
```
#To visualize this we can take top Collaboration between actor and director
director_actor_counts = df.groupby(['director', 'cast'])['title'].count().reset_index()
director_actor_counts = director_actor_counts[director_actor_counts['cast'] != 'Unknown Cast']
director_actor_counts = director_actor_counts[director_actor_counts['director'] != 'Unknown Director']
director_actor_counts = director_actor_counts.rename(columns={'title': 'count'})
director_actor_counts = director_actor_counts.sort_values('count', ascending=False)
top_collaborations = director_actor_counts.head(20)
plt.figure(figsize=(15, 5))
sns.barplot(x='count', y='director', hue='cast', data=top_collaborations)
plt.title('Top Collaborations between Directors and Actors')
plt.xlabel('Count')
plt.ylabel('Director')
```

```
Text(0, 0.5, 'Director')
```



The above graph gives the information of the Director Lors Von Trier has worked maximum no of times with the Stelan and Christian. But this will not provide a clear count and the name from the top 20 collaborators. So, it is better to use a heatmap to get the exact correlation between Top 20 actor and director.

```
#5. To get a heat map, To find the correlation between top 20 Director and cast
plt.figure(figsize=(12, 10))
sns.heatmap( top_collaborations.pivot_table(index='director', columns='cast', values='count').fillna(0),
    annot=True,
    cmap='rainbow')
plt.title('Top Director-Actor Collaborations')
plt.xlabel('Actor')
plt.ylabel('Director')
plt.xticks(rotation=90, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```



The above heat map clearly gives the count count of each director and how many times they have worked with that particular cast. From this we can infer that,

1. Out of top20 Actors and Directors, Lors Von Trier has worked with Charlotte Gainsbourg, Christian Slater, Shia LaBeouf, Sophie, Stacy, Stellan and Uma Thurman a maximum no of times. That is 33 times
2. From this Heatmap, we can get to know that from the top 20 collaborations. Directors Rajmouli and Air Folman has the least count of 21 with their respective actors.

```
#To get to know the director and what type of genre they worked in

df.groupby('director')['listed_in'].value_counts(sort=True, ascending=False)
```

director	listed_in	count
A. L. Vijay	Comedies	14
	International Movies	14
	Sci-Fi & Fantasy	8
	Dramas	6
A. Raajdheep	Dramas	5
...	...	...
Ömer Faruk Sorak	Romantic Movies	8
Şenol Sönmez	Comedies	16
	International Movies	16
	Dramas	8
	Romantic Movies	8

12132 rows × 1 columns

```
colormaps = plt.colormaps() # To know the colors that we can use it.
print(colormaps)
```

```
['magma', 'inferno', 'plasma', 'viridis', 'cividis', 'twilight', 'twilight_shifted', 'turbo', 'berlin', 'managua', 'vanimo', 'Blues
```

```
#6. Which actor works most over all, irrespective of the type
df['cast'].value_counts()
```

cast	count
Unknown Cast	2149
Liam Neeson	161
Alfred Molina	160
John Krasinski	139
Salma Hayek	130
...	...
Kelis	1
Elias Toufexis	1
Claudia Christian	1
Joe Flanigan	1
Bianca Brigitte VanDamme	1

36440 rows × 1 columns

From this we get to know that, excluding unknown actor, Liam Neeson has been productive actor

```
#7. Which Movie actor is more productive
Movie_actor = df[df['type'] == 'Movie']['cast'].value_counts()
Movie_actor
```

cast	count
Unknown Cast	1331
Liam Neeson	161
Alfred Molina	157
John Krasinski	138
Salma Hayek	130
...	...
Albert Hall	1
William Hickey	1
Karyn Parsons	1
Marion Van Cuyck	1
Heather Lawless	1

25952 rows × 1 columns

From the above, we can infer that excluding the Unknown cast, Liam Neeson is the most productive Movie actor.

```
Tv_actor = df[df['type'] == 'TV Show']['cast'].value_counts()
Tv_actor
```

cast	count
Unknown Cast	818
David Attenborough	82
Takahiro Sakurai	56
Yuki Kaji	45
Ai Kayano	41
...	...
Jamie Davis	1
Marley Dias	1
Mae Elliessa	1
Mason Wells	1
Zaela Rae	1

14864 rows × 1 columns

From the above info, we can say that,

1. David Attenborough is the most productive TV actor, excluding the Unknown cast.

```
#8. Top10 genre
top_genres = df['listed_in'].value_counts()
top_genres
```

listed_in	count
Dramas	29806
International Movies	28243
Comedies	20829
International TV Shows	12845
Action & Adventure	12216
Independent Movies	9834
Children & Family Movies	9771
TV Dramas	8942
Thrillers	7107
Romantic Movies	6412
TV Comedies	4963
Crime TV Shows	4733
Horror Movies	4571
Kids' TV	4568
Sci-Fi & Fantasy	4037
Music & Musicals	3077
Romantic TV Shows	3049
Documentaries	2409
Anime Series	2313
TV Action & Adventure	2288
Spanish-Language TV Shows	2126
British TV Shows	1808
Sports Movies	1531
Classic Movies	1443
TV Mysteries	1281
Korean TV Shows	1122
Cult Movies	1077
Anime Features	1045
TV Sci-Fi & Fantasy	1045
TV Horror	941
Docuseries	845
LGBTQ Movies	838
TV Thrillers	768
Teen TV Shows	742
Reality TV	735
Faith & Spirituality	719
Stand-Up Comedy	540
Movies	412
TV Shows	337
Classic & Cult TV	272
Stand-Up Comedy & Talk Shows	268
Science & Nature TV	157

From the netflix data, we get to know that Dramas Genre is highest count and the least is the Science and Nature TV type. So we can infer that, people mostly like the Drama, International movies, Comedies the most than the Science and Nature, Stand up comedy, Classic and Cult TV.

So based on the people interest, we can increase the no of movies or Tv shows based on the Genre like Drama, International movies and Comedies.

```
#9. Which genres are predominantly represented in TV Shows vs Movies?
movie_genres = df[df['type'] == 'Movie']['listed_in'].value_counts().index[:10]
tv_show_genres = df[df['type'] == 'TV Show']['listed_in'].value_counts().index[:10]

print("Top 10 Movie Genres:", movie_genres)
print("Top 10 TV Show Genres:", tv_show_genres)
```

```
Top 10 Movie Genres: Index(['Dramas', 'International Movies', 'Comedies', 'Action & Adventure',
       'Independent Movies', 'Children & Family Movies', 'Thrillers',
       'Romantic Movies', 'Horror Movies', 'Sci-Fi & Fantasy'],
      dtype='object', name='listed_in')
Top 10 TV Show Genres: Index(['International TV Shows', 'TV Dramas', 'TV Comedies', 'Crime TV Shows',
       'Kids' TV', 'Romantic TV Shows', 'Anime Series',
       'TV Action & Adventure', 'Spanish-Language TV Shows',
       'British TV Shows'],
      dtype='object', name='listed_in')
```

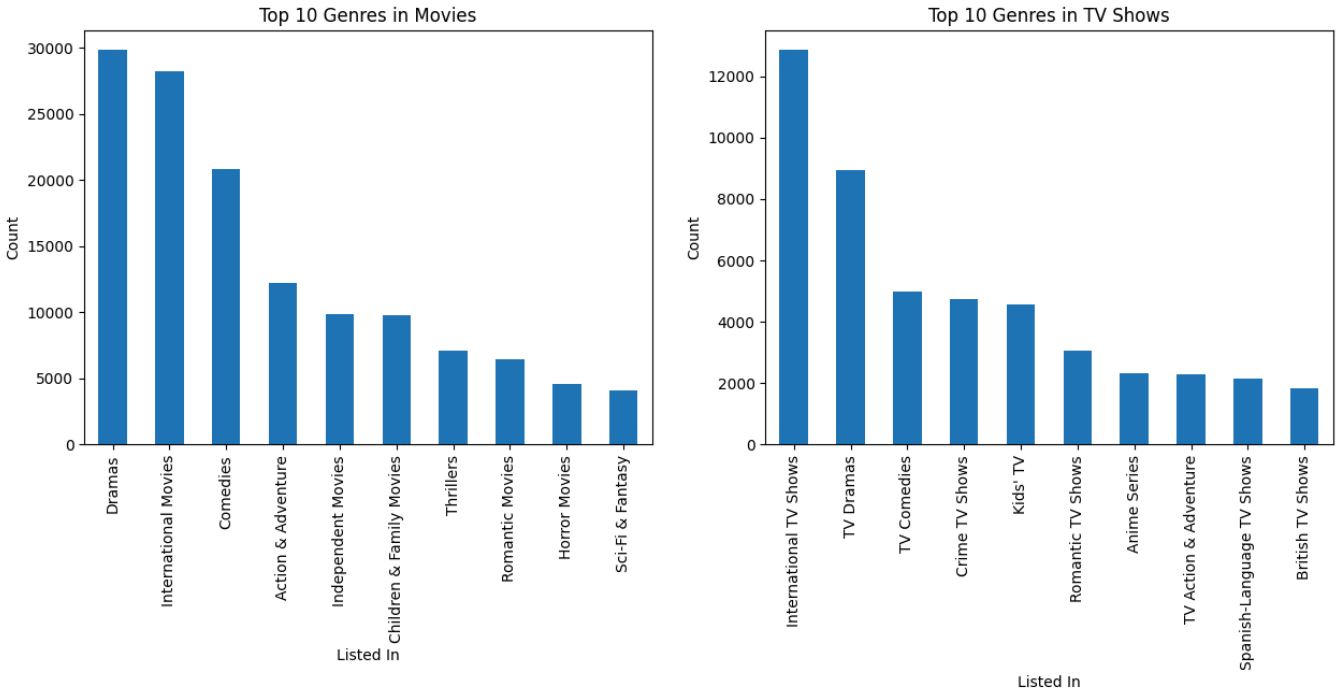
From the above output, we can conclude that,

1. For Movie - 'Dramas' is the top 1 and 'Sci-Fi & Fantasy' is in the 10th position.
2. For TV Show - 'International TV Shows' is the top1 and 'British TV Shows' is in the 10th position.

```
df.columns
```

```
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
       'release_year', 'rating', 'duration', 'listed_in', 'description',
       'Movie_Mins', 'Show_Seasons', 'month_added', 'day_added'],
      dtype='object')
```

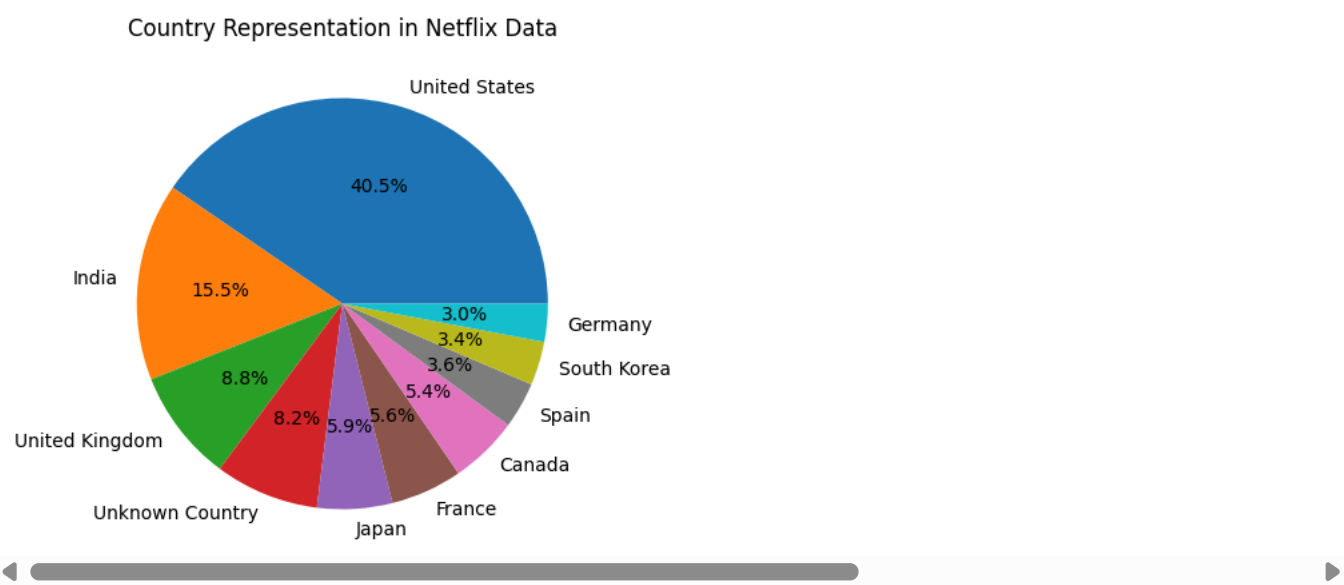
```
#10. Which genres are predominantly represented in TV Shows vs Movies? To show this using visualization.

movie_genres = df[df['type'] == 'Movie']['listed_in'].value_counts().index[:10]
tv_show_genres = df[df['type'] == 'TV Show']['listed_in'].value_counts().index[:10]

#To compare and visualize this, I am using subplots to visulaize both Movie shows and TV shows.side by side.
# Create separate DataFrames for movies and TV shows
movies_df = df[df['type'] == 'Movie']
tv_shows_df = df[df['type'] == 'TV Show']

# Get the top 10 genres for each type
top_movie_genres = movies_df['listed_in'].value_counts().head(10).index
top_tv_show_genres = tv_shows_df['listed_in'].value_counts().head(10).index

# Create a figure and axes for the plots
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Plot for movies
movies_df['listed_in'].value_counts().head(10).plot(kind='bar', ax=axes[0])
axes[0].set_title('Top 10 Genres in Movies')
axes[0].set_xlabel('Listed In')
axes[0].set_ylabel('Count')

# Plot for TV shows
tv_shows_df['listed_in'].value_counts().head(10).plot(kind='bar', ax=axes[1])
axes[1].set_title('Top 10 Genres in TV Shows')
axes[1].set_xlabel('Listed In')
axes[1].set_ylabel('Count')

plt.show()
```



From the above bar plot, We can easily read and understand the count of movies in top 10 genres and count of Tv shows in Top 10 genres. Based on this we can say

1. People who love to watch movies more likely to see The 'Dramas' genre then comes the 'International Movies' and then comes the 'Comedies'
2. People who love to watch TV shows most likely to watch the Genre of type 'International TV Shows' then comes the 'TV Dramas' and so on. Just mentioned the top 2

```
#11. To check, How diverse is Netflix's data in terms of country representation?
df['country'].value_counts()

#Representing this using pie chart, gives the percentage of distribution across various countries
plt.figure(figsize=(15, 5))
df['country'].value_counts().head(10).plot(kind='pie', autopct='%1.1f%%')
plt.title('Country Representation in Netflix Data')
plt.ylabel('')
plt.show()
```

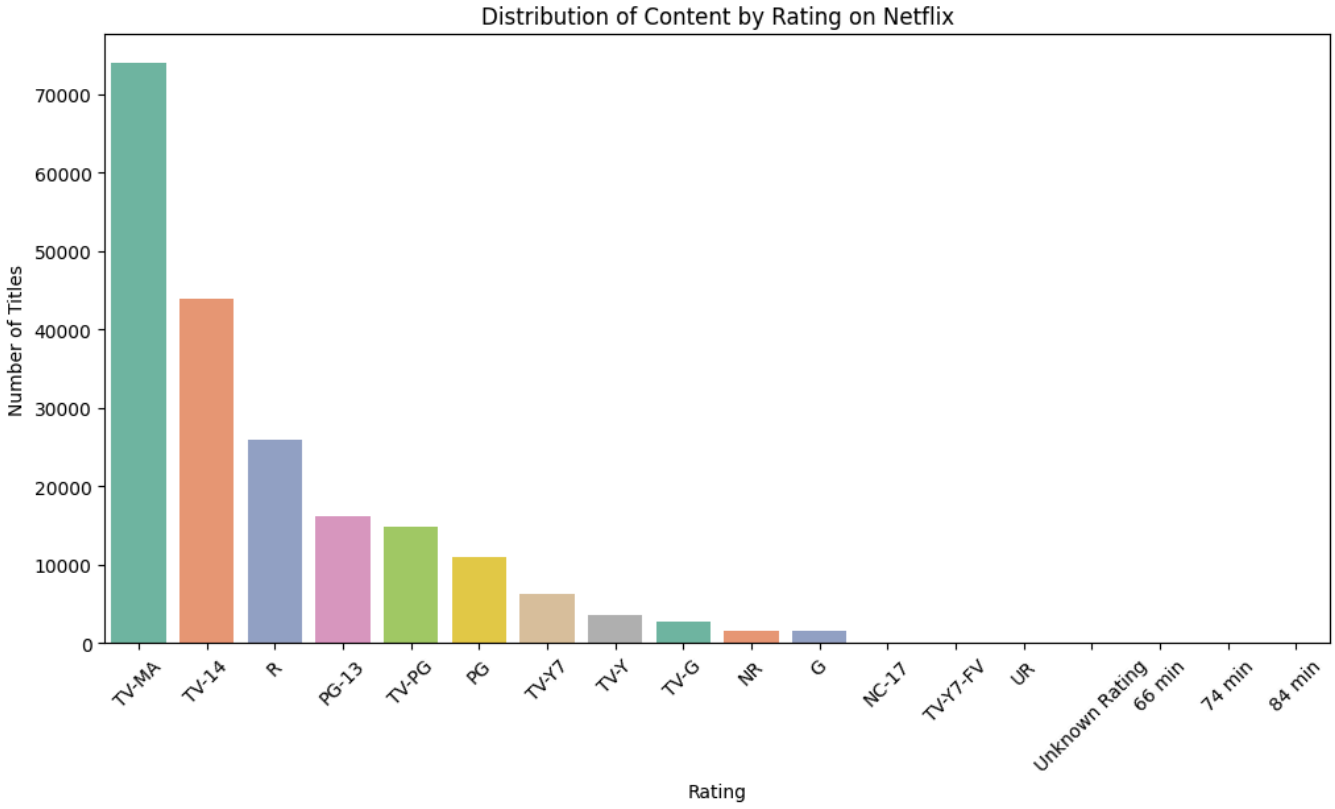


From the above pictorial representation,

1. We can see that Movies/ Tv shows is Highest in United states then comes India.
2. We can also say that 40% of the data belongs to United States and rest is shared among other countries.

```
#12. What is the distribution of content across different ratings (TV-MA, PG, etc.)?

rating_counts = df['rating'].value_counts().reset_index()
rating_counts.columns = ['rating', 'count']

# Plotting
plt.figure(figsize=(12, 6))
sns.barplot(data=rating_counts, x='rating', y='count', palette='Set2')
plt.title('Distribution of Content by Rating on Netflix')
plt.xticks(rotation=45)
plt.ylabel('Number of Titles')
plt.xlabel('Rating')
```

```
<ipython-input-150-d1c03aefc2af>:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `le

  sns.barplot(data=rating_counts, x='rating', y='count', palette='Set2')
Text(0.5, 0, 'Rating')
```



From this we can say that the rating 'TV-MA' has more no of movies with this rating.Then comes the rating 'TV-14'.

1. Mature audience rating movies count is highest compared to any other kind of rating.

```
# 13. Which types are more genre-diverse among TV shows and Movies
movie_genres = df[df['type'] == 'Movie']['listed_in'].value_counts()
tv_show_genres = df[df['type'] == 'TV Show']['listed_in'].value_counts()
movie_genres.shape, tv_show_genres.shape
```

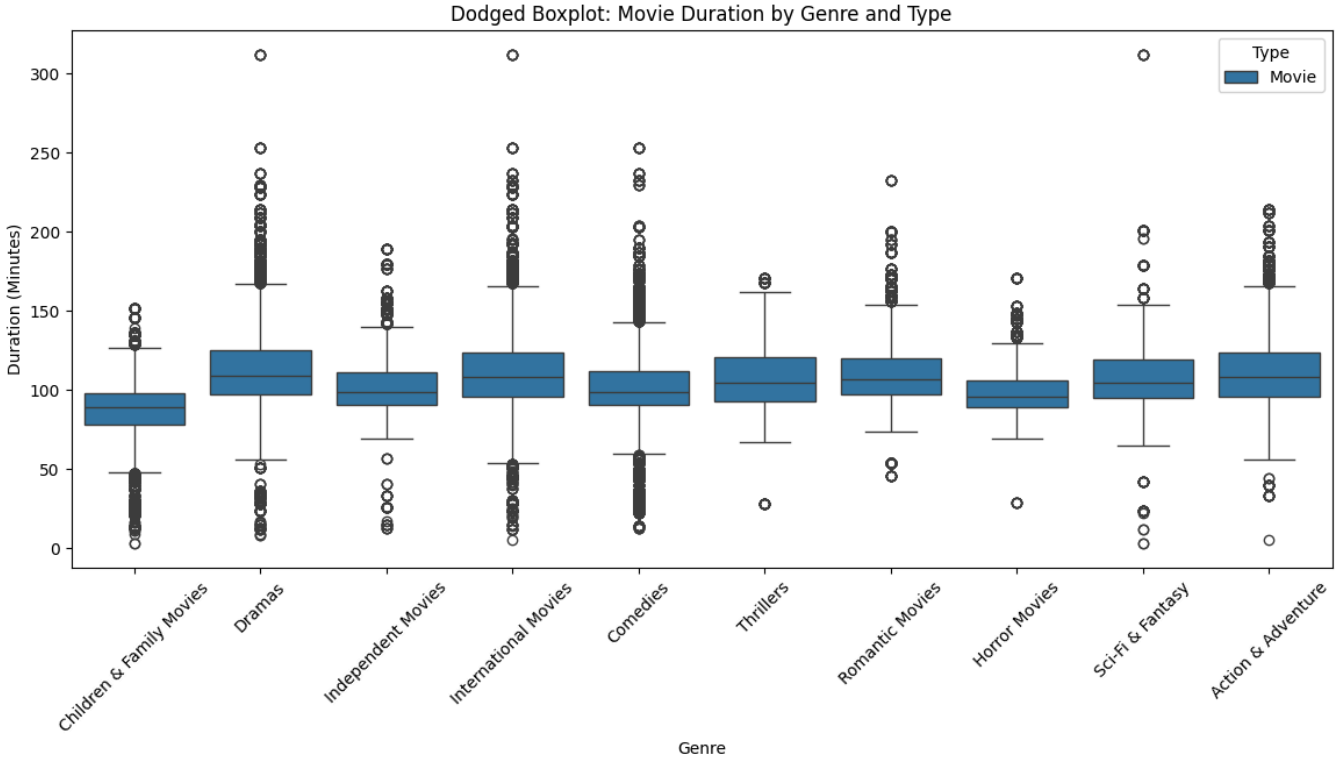
```
((20,), (22,))
```

From this we can say that TV show genres are slightly more compared to the Movies type. Based on the people interest, we can say that people who watches TV shows have more variety of Genres to watch than that of Movies.

```
#14. Is there any extremes / outliers for the duration of Movies based on Top Genres

#Considering only top 10 lengthier movies in top genres and finding the outliers
df_movie = df[df['type'] == 'Movie'].copy()

# Filter top genres (optional to keep plot readable)
top_genres = df_movie['listed_in'].value_counts().nlargest(10).index
df_movie = df_movie[df_movie['listed_in'].isin(top_genres)]

# Plot dodged boxplot
plt.figure(figsize=(14, 6))
```

```
sns.boxplot(data=df_movie, x='listed_in', y='Movie_Mins', hue='type')

plt.title('Dodged Boxplot: Movie Duration by Genre and Type')
plt.xlabel('Genre')
plt.ylabel('Duration (Minutes)')
plt.xticks(rotation=45)
plt.legend(title='Type')
```

```
<matplotlib.legend.Legend at 0x7e4da73f2890>
```



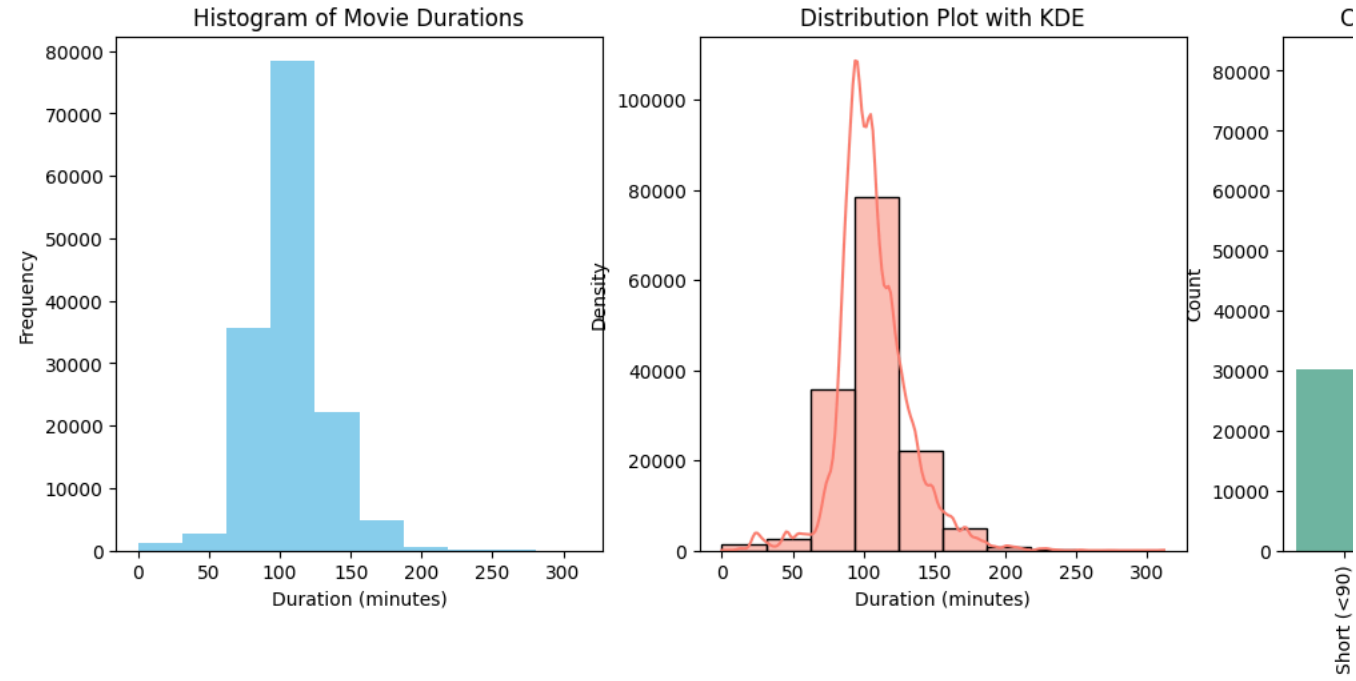
From this we can say that,

Genres like , 'International Movies' , 'Comedies' , 'Dramas' have more outliers compared to others. Thrillers Genre have the least outliers.

```
#15. Univariate analysis on a Movie duration using histogram, distribution plot and a bar graph together as sub plots.

# Here, I have analysed the Movie mins or duration of a movie with the frequency.
# To check the plot where the most movies fall within the what range.
plt.figure(figsize=(16, 5))

# Histogram
plt.subplot(1, 3, 1)
plt.hist(df['Movie_Mins'], bins=10, color='skyblue')
plt.title('Histogram of Movie Durations')
plt.xlabel('Duration (minutes)')
plt.ylabel('Frequency')

# Using KDE to check where exactly the peak is? and we can find the max count.

# Distribution Plot with KDE
plt.subplot(1, 3, 2)
sns.histplot(df['Movie_Mins'], kde=True, bins=10, color='salmon')
plt.title('Distribution Plot with KDE')
plt.xlabel('Duration (minutes)')
plt.ylabel('Density')


# Count Plot (Categorized Durations)
# Bin the durations into categories
bins = [0, 90, 120, 150, 200]
labels = ['Short (<90)', 'Medium (90-120)', 'Long (120-150)', 'Very Long (>150)']
df['duration_category'] = pd.cut(df['Movie_Mins'], bins=bins, labels=labels)

plt.subplot(1, 3, 3)
sns.countplot(x='duration_category', data=df, palette='Set2')
plt.title('Count Plot of Duration Categories')
plt.xlabel('Duration Category')
```

```
plt.ylabel('Count')
plt.xticks(rotation=90)
```

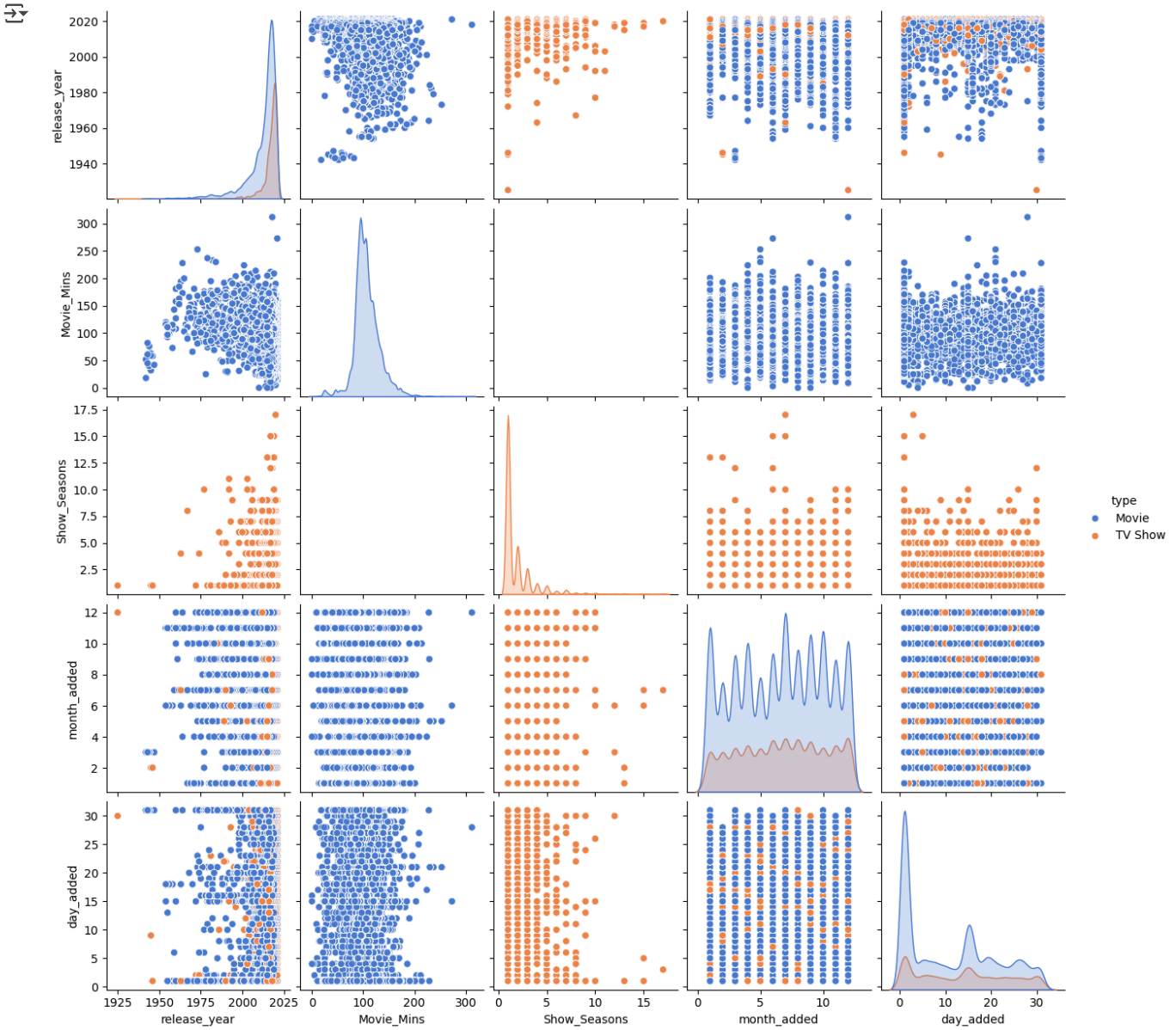
```
<ipython-input-153-80c1e5d3d17a>:31: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `le

  sns.countplot(x='duration_category', data=df, palette='Set2')
([0, 1, 2, 3],
 [Text(0, 0, 'Short (<90)'),
  Text(1, 0, 'Medium (90-120)'),
  Text(2, 0, 'Long (120-150)'),
  Text(3, 0, 'Very Long (>150)')])
```



From the above subplots, we can conclude that,

1. Histogram: Most movies fall within the 60-150 minute range. Highest for the 100 mins.
2. Distribution Plot (with KDE): The KDE curve shows a clear peak around 95-100 minutes.
3. Count Plot (Categorized Durations): The majority of movies are in the "Medium (90-120)" category. Very few movies are "Very Long" (>150 minutes), supporting the trend toward shorter content.

```
#Pair plot
sns.pairplot(df, hue='type', palette = 'muted')
plt.show()
```



Insights:

1. There's a general upward trend in the number of movies released over the years. The peak was in 2018.
2. There was a slight decline after 2018.
3. The United States has the most movies and TV shows.
4. India is in second place for movies.
5. The UK is third for movies and TV shows combined.
6. More movies are added in July, followed by January and December.
7. The fewest additions are in the first few months of the year and in the summer.
8. TV shows have a similar pattern, with the lowest releases in February.
9. For movies, the top genres are international movies, dramas, and comedies.
10. For TV shows, the top genres are international TV shows, TV dramas, and crime TV shows.
11. Reveals that certain genres, such as documentaries and stand-up comedy, tend to have longer durations compared to others.
12. The count plot shows the distribution of movie durations based on categorized duration groups (e.g., short, medium, long).
13. As there is no relationship between TV shows and movies. So there is an empty graph.

Recommendations from Business Case Study for Netflix.

1. Movies/ TV Shows released is highest in the year 2018 and slightly getting decreased in 2019, 2020 and 2021. Netflix should release good content Movies/TV Shows and increase the no of shows to get released per year, when compared to previous years. The Trend or a graph should be growing over the time.
2. Increasing the no of Movies/ TV Shows increases the revenue of Netflix.
3. From the provided data, we found that, there is a greater number of Movies than the TV Shows. It is less than half of the count of Movies. So, even the TV shows count should be increased to increase the overall profit.
4. We found that the maximum no of movies have the duration of 94 mins. By increasing the duration of movies and making the user to engage watching it. Can also be beneficial.
5. If the user likes the content, They will be subscribing. So need to release the shows where users like the most.
6. Maximum no of TV Shows have the count of 13 seasons. Increasing in no of seasons will also be beneficial to attract more users to get subscribed , which intact leads to good profit.
7. While doing the country wise study for movies and Tv shows count. It was found that most of the countries have no Movies or TV shows. If the country has either the Movies or TV shows, then its vice versa that we can also have TV Shows / Movies accordingly. To increase the more no of viewers and allowing them to engage watching it. Which is the major thing, that should be focused on.
8. It is a common thing that, If there are people who loves to watch movies, there can be at least a probability that people will also love to watch TV Shows. So, from this we can understand and implement to add more TV shows and even increasing the no of Seasons.
9. The best time from the study is, More no of movies are released on Jan 1st , on account of New year and I would consider this as holiday. More no of movies are added on Holidays rather than any of the other days.
10. Taking the count of movies, more no of movies are added in the month of July , then comes the Jan
11. We could see that more no of TV shows are added in the month of December. As there will be lot of holidays in the month of Dec. Likewise, There should be proper plan about upcoming holidays and based on that movies or shows should be added to attract the viewers or the audience.
12. Based on the study, we gathered information on directors and the number of movies or shows they have directed within each genre. This analysis helps us identify which directors are most active or influential in specific genres, such as drama, comedy, action, or thriller etc. By understanding the genre preferences and strengths of each director, we can make more informed decisions on content acquisition, collaboration and marketing strategies
13. Few of the directors have worked for only 1 type of genre, need to promote them to work on under rated Genre like Sci-Fi . So that we can promote the talent of Director as well and even increase the count of Genres. This will also attract viewers wishlist.
14. From the above data, it is not much clear regarding the top hits of Movies or shows. But according to my understanding, As there are more no of movies rather than TV Shows , so those movies will have Top hits. So with this , we can encourage the Movie directors with same genre and with similar stories should be made as a TV shows with more episodes. Which will also provide good result.

Double-click (or enter) to edit

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

.....

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

.....

Start coding or generate with AI.

Start coding or generate with AI.